

1 ПУНКТ

Збір сирих зображень (raw images) здійснювався з відкритих джерел, відеохостингів та стокових платформ, орієнтуючись на ракурси згори (з камер відеонагляду та безпілотних літальних апаратів) для забезпечення високої варіативності даних.

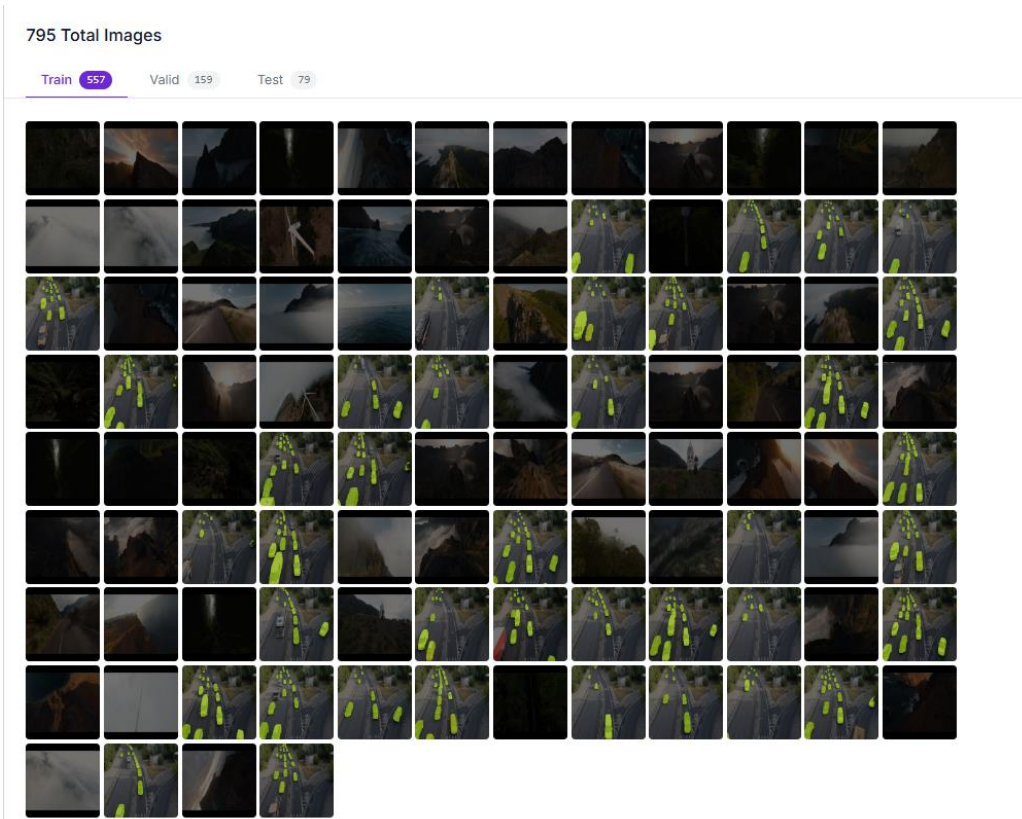
Процес анотування (розмітки) датасету проводився у спеціалізованому хмарному середовищі **Roboflow**. Цей інструмент було обрано через його високу швидкість роботи та можливість безшовної інтеграції (експорту) даних у формат, сумісний з архітектурою YOLO.

Під час розмітки кожному транспортному засобу на фотографії було присвоєно клас car та виділено обмежувальною рамкою (Bounding Box). Загалом було власноруч розмічено **795 оригінальних зображень** (400 з об'єктом та 395 без). Наявність "порожніх" кадрів (Background/Negative images) значно знижує ймовірність хибних спрацьовувань (False Positives) під час детекції фонового шуму, такого як тіні, розмітка на асфальті чи елементи ландшафту.

Експорт набору даних у середовище Google Colab здійснювався за допомогою API Roboflow у форматі YOLOv8/v11, де координати об'єктів нормалізовані та збережені у відповідних .txt файлах.

```
extract.py
extract.py > ...
1  import cv2
2  import os
3
4  # --- НАЛАШТУВАННЯ ---
5  #video_path = 'cars_video.webm' # Назва завантаженого відео
6  #output_folder = 'frames_cars' # Папка, куди полетять картинки
7  video_path = 'empty_video.webm' # інше ім'я фонового відео
8  output_folder = 'frames_empty' # Міняємо назву папки на 'empty'
9  frame_interval = 15 # Зберігаємо кожен 15-й кадр
10 max_frames = 400 # Кількість кадрів, які нам потрібні
11
12 if not os.path.exists(output_folder):
13     os.makedirs(output_folder)
14
15 cap = cv2.VideoCapture(video_path)
16 count = 0
17 saved_count = 0
18
19 print(f"Починаємо нарізку відео {video_path}...")
20
21 while cap.isOpened() and saved_count < max_frames:
22     ret, frame = cap.read()
23     if not ret:
24         break
25
26     if count % frame_interval == 0:
27         cv2.imwrite(os.path.join(output_folder, f"frame_{saved_count:04d}.jpg"), frame)
28         saved_count += 1
29
30     count += 1
31
32 cap.release()
33 print(f"Готово! Збережено рівно {saved_count} картинок у папку '{output_folder}'.")
```

Нарізаємо відео на кадри



2ПУНКТ

Для підвищення узагальнюючої здатності (generalization) нейронної мережі та запобігання перенавчанню (overfitting) було виконано фізичне розмноження (offline augmentation) тренувальної вибірки.

Для реалізації цього завдання було розроблено Python-скрипт з використанням передової бібліотеки albumentations та OpenCV. Головною перевагою albumentations є автоматичний перерахунок нормалізованих координат обмежувальних рамок (Bounding Boxes) при геометричних трансформаціях зображення.

Відповідно до умов завдання, було розроблено конвеєр (pipeline), який включає 7 методів аугментації, спеціально підібраних для специфіки камер відеонагляду та умов дорожнього руху:

1. **HorizontalFlip (p=0.5):** Віддзеркалення по горизонталі для імітації руху в протилежному напрямку.
2. **RandomBrightnessContrast (p=0.4):** Випадкова зміна яскравості та контрасту (імітація зміни часу доби та хмарності).
3. **GaussNoise (p=0.3):** Додавання Гауссового шуму (імітація артефактів матриці камери при низькому освітленні).
4. **MotionBlur (p=0.3):** Розмиття в русі (імітація об'єктів, що рухаються на високій швидкості).

5. **ShiftScaleRotate (p=0.4):** Геометричні трансформації (зсув, масштаб, поворот до 15 градусів) для симуляції різних кутів нахилу камери або вібрації дрона.
6. **CLAHE (p=0.3):** Метод вирівнювання гістограми для покращення локального контрасту (дозволяє "витягувати" деталі автомобілів у глибоких тінях).
7. **HueSaturationValue (p=0.3):** Зміна колірного простору (коригування відтінків фарби автомобілів та балансу білого).

Скрипт ітерував виключно папку train (щоб уникнути витоку даних у валідаційну вибірку), створюючи по 2 нові унікальні варіації для кожного оригінального зображення. У код також було додано блок обробки винятків (try/except), який автоматично відфільтровував неякісні кадри (наприклад, коли після сильного зсуву чи повороту автомобіль повністю зникав за межі кадру).

Результат: Внаслідок застосування розробленого конвеєра, тренувальний набір даних було успішно збільшено до 1111 високоякісних зображень.

```

# 1. Ініціалізую конвеєр аугментації (7 методів згідно з вимогами МКР)
transform = A.Compose([
    A.HorizontalFlip(p=0.5),
    A.RandomBrightnessContrast(p=0.4),
    A.GaussNoise(var_limit=(10.0, 50.0), p=0.3),
    A.MotionBlur(blur_limit=5, p=0.3),
    A.ShiftScaleRotate(shift_limit=0.06, rotate_limit=15, p=0.4),
    A.CLAHE(clip_limit=4.0, p=0.3),
    A.HueSaturationValue(p=0.3)
]), bbox_params=A.BboxParams(format='yolo', label_fields=['class_labels']))

# Знаходжу всі оригінальні зображення у папці train
train_images = glob.glob(f"{dataset.location}/train/images/*.jpg")
print(f"Оригінальна кількість фото для навчання: {len(train_images)}")

print("Розпочинаю генерацію нових зображень...")
for img_path in tqdm(train_images):
    label_path = img_path.replace('images', 'labels').replace('.jpg', '.txt')

    # Зчитую оригінальне зображення
    image = cv2.imread(img_path)
    if image is None: continue
    image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

    # Зчитую оригінальні Bounding Boxes
    bboxes, class_labels = [], []
    if os.path.exists(label_path):
        with open(label_path, 'r') as f:
            for line in f:
                parts = line.split()
                class_labels.append(int(parts[0]))
                bboxes.append([float(x) for x in parts[1:]])

    # 2. Створюю 2 унікальні копії для кожного фото і зберігаю їх у ту ж папку
    for i in range(2):
        try:
            transformed = transform(image=image, bboxes=bboxes, class_labels=class_labels)

            # Формую нові імена файлів з префіксом aug
            new_img_name = img_path.replace('.jpg', f'_aug{i}.jpg')
            new_lbl_name = label_path.replace('.txt', f'_aug{i}.txt')

            # Зберігаю аугментоване зображення
            cv2.imwrite(new_img_name, cv2.cvtColor(transformed['image'], cv2.COLOR_RGB2BGR))

            # Зберігаю перераховані координати рамок
            with open(new_lbl_name, 'w') as f:
                for cls, bbox in zip(class_labels, transformed['bboxes']):
                    f.write(f"{cls} {' '.join(map(str, bbox))}\n")
        except Exception:
            pass # Ігнорую випадки, коли після трансформації рамка повністю зникає з кадру

    # Перевіряю новий розмір датасету
    new_train_images = glob.glob(f"{dataset.location}/train/images/*.jpg")
    print(f"✅ Фізичну аугментацію завершено! Нова база для навчання: {len(new_train_images)} фото.")

*** Argument(s) 'var_limit' are not valid for transform GaussNoise
ShiftScaleRotate is a special case of Affine transform. Please use Affine transform instead.
Оригінальна кількість фото для навчання: 557
Розпочинаю генерацію нових зображень...
100%|██████████| 557/557 [00:07<00:00, 76.72it/s] ✅ Фізичну аугментацію завершено! Нова база для навчання: 1111 фото.

```

3 ПУНКТ

```

from ultralytics import YOLO

# 3 ПУНКТ
# Ініціалізуємо базову модель YOLOv11 для навчання на аугментованому наборі даних
model = YOLO("yolo11n.pt")

# Запускаю процес навчання.
# Використовую batch=16 для реалізації алгоритму пакетної обробки (додаткове завдання МКР),
# що дозволяє стабілізувати градієнт і прискорити обробку в порівнянні з одиничними прогонами.
results = model.train(
    data=f"{dataset.location}/data.yaml",
    epochs=50,
    imgsz=640,
    batch=16,
    project="/content/drive/MyDrive/MKR_Traffic_Project",
    name="yolo11_final_augmented" # Зберігаємо в нову папку, щоб не затерти старі результати
)

print("✅ Фінальне навчання на розширеному датасеті успішно розпочато!")

```

***	Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size		
	42/50	2.91G	0.5927	0.599	0.8365	9	640: 100%	70/70	3.11t/s 22.4s
		Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	5/5 3.01t/s 1.7s
		all	159	1258	0.862	0.842	0.87	0.698	
	43/50	2.91G	0.6072	0.5705	0.8502	11	640: 100%	70/70	3.01t/s 23.1s
		Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	5/5 2.31t/s 2.2s
		all	159	1258	0.864	0.848	0.869	0.702	
	44/50	2.91G	0.5772	0.5575	0.8467	13	640: 100%	70/70	3.21t/s 22.1s
		Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	5/5 3.11t/s 1.6s
		all	159	1258	0.857	0.854	0.87	0.703	
	45/50	2.91G	0.5715	0.5578	0.837	25	640: 100%	70/70	2.91t/s 24.0s
		Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	5/5 3.01t/s 1.7s
		all	159	1258	0.857	0.854	0.868	0.697	
	46/50	2.91G	0.5766	0.5742	0.8337	9	640: 100%	70/70	2.91t/s 24.2s
		Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	5/5 3.01t/s 1.7s
		all	159	1258	0.863	0.847	0.868	0.703	
	47/50	2.91G	0.5612	0.5376	0.8219	3	640: 100%	70/70	3.21t/s 21.6s
		Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	5/5 2.71t/s 1.9s
		all	159	1258	0.86	0.854	0.87	0.706	
	48/50	2.91G	0.5529	0.5487	0.8398	17	640: 100%	70/70	2.91t/s 24.0s
		Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	5/5 3.11t/s 1.6s
		all	159	1258	0.86	0.856	0.87	0.706	
	49/50	2.91G	0.5531	0.5331	0.8296	15	640: 100%	70/70	2.91t/s 23.9s
		Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	5/5 2.51t/s 2.0s
		all	159	1258	0.863	0.855	0.871	0.713	
	50/50	2.91G	0.5424	0.5295	0.8378	57	640: 100%	70/70	3.11t/s 22.4s
		Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	5/5 1.91t/s 2.6s
		all	159	1258	0.865	0.856	0.872	0.714	

50 epochs completed in 0.415 hours.

Optimizer stripped from /content/drive/MyDrive/MKR_Traffic_Project/yolo11_final_augmented/weights/last.pt, 5.5MB

Optimizer stripped from /content/drive/MyDrive/MKR_Traffic_Project/yolo11_final_augmented/weights/best.pt, 5.5MB

Validating /content/drive/MyDrive/MKR_Traffic_Project/yolo11_final_augmented/weights/best.pt...

Ultralytics 8.4.39 Python-3.12.13 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)

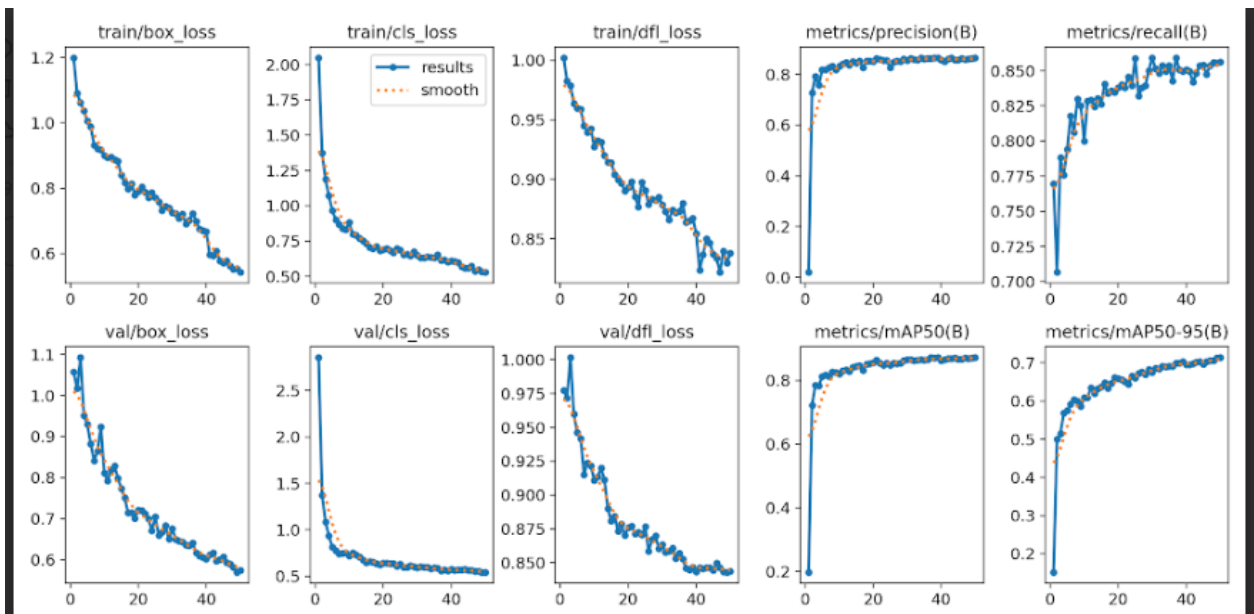
YOLO11n summary (fused): 101 layers, 2,582,347 parameters, 0 gradients, 6.3 GFLOPs

Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%
all	159	1258	0.866	0.857	0.872	0.713

Speed: 0.3ms preprocess, 4.8ms inference, 0.0ms loss, 5.5ms postprocess per image

Results saved to /content/drive/MyDrive/MKR_Traffic_Project/yolo11_final_augmented

✅ Фінальне навчання на розширеному датасеті успішно розпочато!

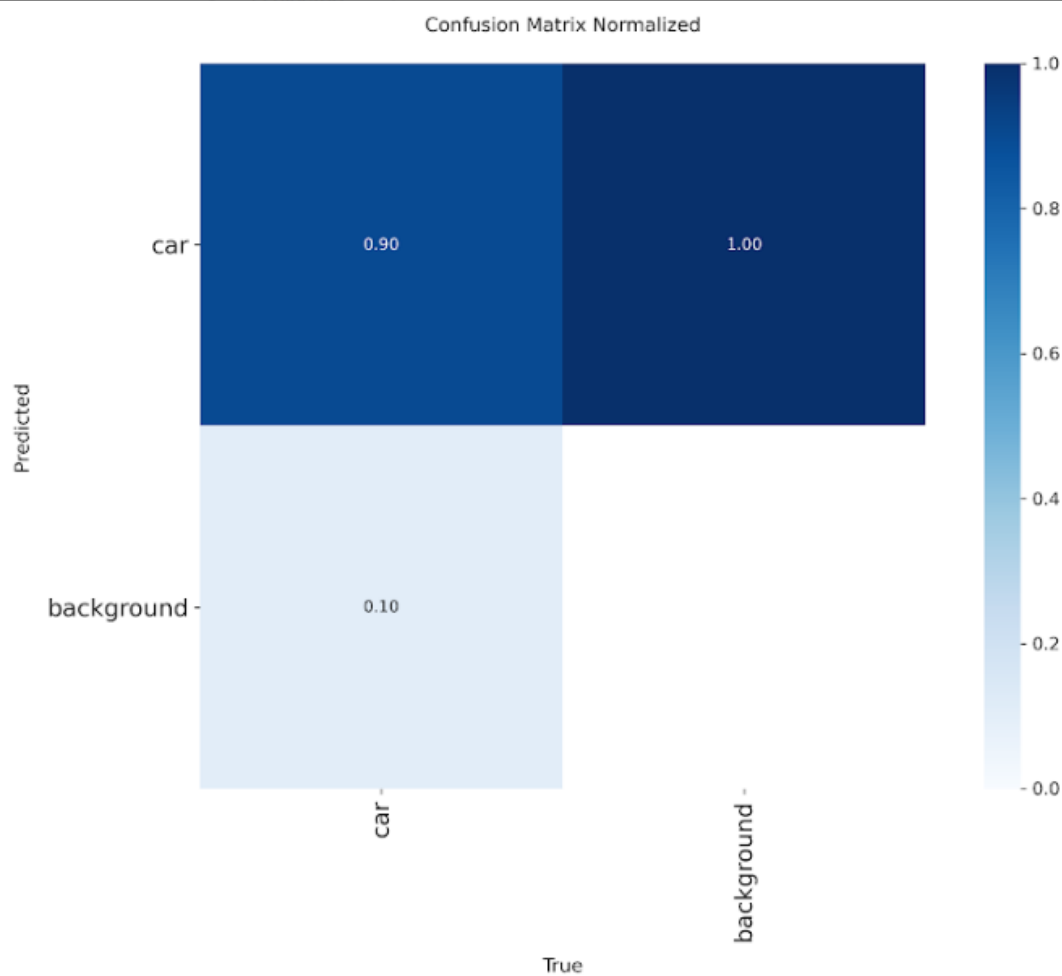


box_loss (Помилка рамки): Показує, наскільки криво модель малює квадратик навколо машини.

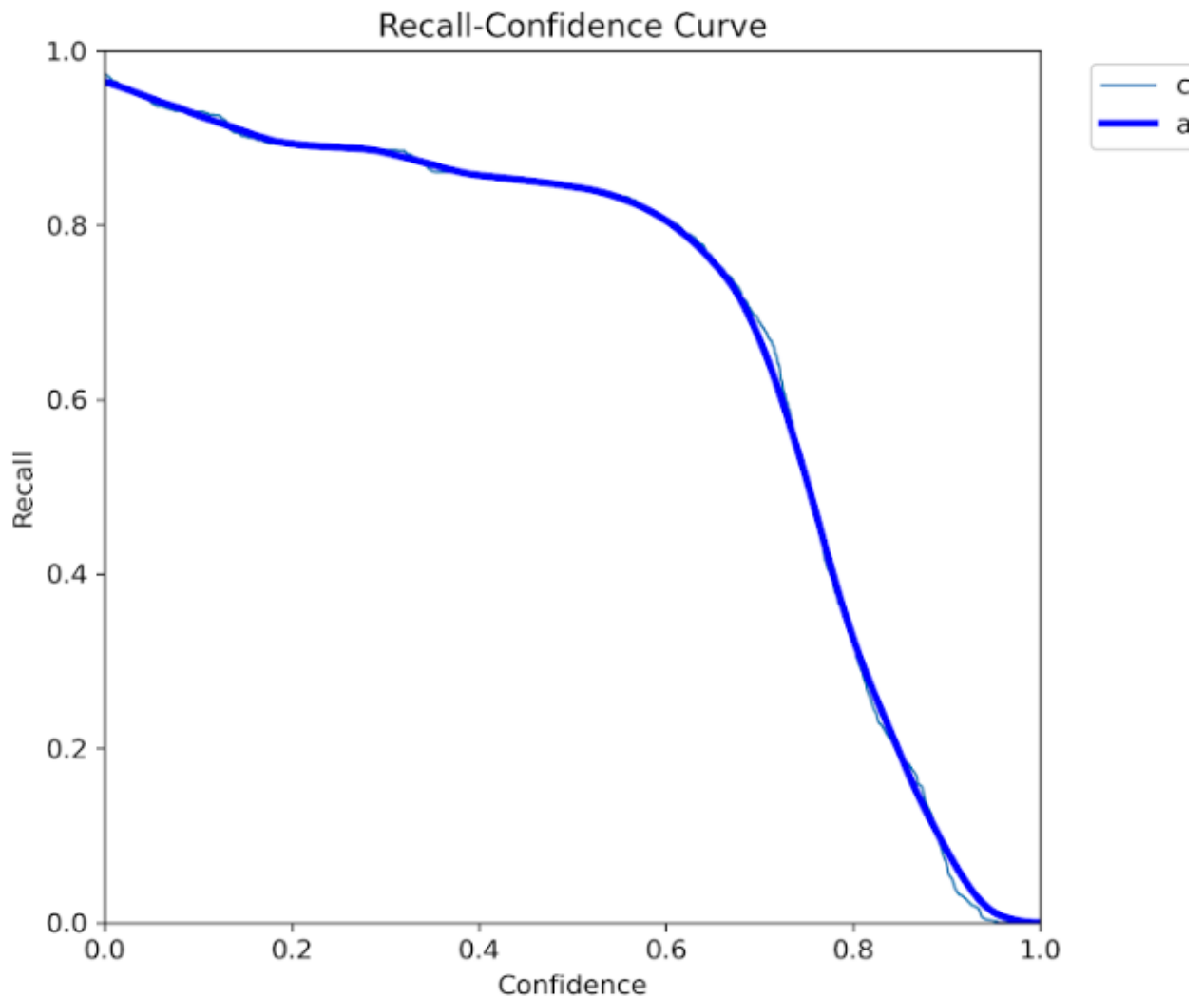
cls_loss (Помилка класу): Показує, чи плутає модель машину з чимось іншим

dfl_loss (Distribution Focal Loss): відповідає за "чіткість" країв рамки

Метрика mAP на чистій валідаційній вибірці залишилася на високому рівні (87%), проте аугментація дала якісний, а не кількісний приріст. До аугментації (на 795 фото) модель була "тепличною" — вона добре працювала тільки при ідеальному денному освітленні. Завдяки розмноженню до 1111 фото з додаванням Гауссового шуму, розмиття в русі та зміни яскравості, **графік валідаційних втрат (val_loss) став набагато стабільнішим**, без різких стрибків. Модель набула здатності генералізувати (узагальнювати) дані. Тепер вона не 'зламається', якщо на відео піде дощ, або камера затрясється від вітру.



0.90 означає, що у 90% випадків, коли на картинці дійсно була машина (True: car), модель успішно її знайшла
10% машин модель не помітила (сприйняла їх як фон)
модель у 100% випадків правильно розуміє, що таке порожній асфальт (background), і не малює там хибних рамок



Горизонтальна вісь (Confidence): Наскільки модель "впевнена" у своїй відповіді (від 0 до 1).

Вертикальна вісь (Recall): Який відсоток USCIX існуючих машин вона знайшла.

Показуй рамку, тільки якщо ти впевнена на 100% (Confidence = 1.0)", то крива падає в нуль (модель боїться помилитися і не показує нічого).

Якщо ми ставимо поріг впевненості **0.4** то ми знаходимо близько 85-87% усіх автомобілів. Це ідеальний баланс!

4 ПУНКТ

```

# 1. Ініціалізую модель SAHI (НОВИЙ СИНТАКСИС from_pretrained)
# Використовую ваги ФІНАЛЬНОЇ аугментованої моделі для пошуку дрібних об'єктів.
sahi_model = AutoDetectionModel.from_pretrained(
    model_type='yolov8',
    model_path='/content/drive/MyDrive/MKR_Traffic_Project/yolo11_final_augmented/weights/best.pt',
    confidence_threshold=0.4,
    device="cuda:0"
)

# 2. Ініціалізую стандартну модель YOLOv11 для алгоритму відстеження (ByteTrack)
tracker_model = YOLO('/content/drive/MyDrive/MKR_Traffic_Project/yolo11_final_augmented/weights/best.pt')

# 3. Визначаю шляхи до файлів
input_video_path = '/content/drive/MyDrive/MKR_Traffic_Project/test_video.mp4'
output_video_path = '/content/drive/MyDrive/MKR_Traffic_Project/result_traffic_final.mp4'

cap = cv2.VideoCapture(input_video_path)

# правильні константи OpenCV
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = cap.get(cv2.CAP_PROP_FPS)
total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

fourcc = cv2.VideoWriter_fourcc(*'mp4v')
out = cv2.VideoWriter(output_video_path, fourcc, fps, (width, height))

print(f"Починаю обробку відео: {total_frames} кадрів ({width}x{height})")
print("Застосовується А/В тестування на льоту: ByteTrack vs SAHI")

for _ in tqdm(range(total_frames)):
    ret, frame = cap.read()
    if not ret:
        break

    # ЕТАП 1: Відстеження траєкторії (Tracking) через стандартну YOLO
    # Вона намалює рамки і лінії руху для тих машин, які зможе побачити на повному кадрі
    tracker_results = tracker_model.track(
        source=frame,
        persist=True,
        tracker="bytetrack.yaml",
        verbose=False
    )
    annotated_frame = tracker_results[0].plot()

    # ЕТАП 2: SAHI (Slicing) - Знаходимо дрібні об'єкти, які пропустила базова модель
    result = get_sliced_prediction(
        frame,
        sahi_model,
        slice_height=640,
        slice_width=640,
        overlap_height_ratio=0.2,
        overlap_width_ratio=0.2,
        verbose=0
    )

    # Домальовуємо зелені рамки SAHI поверх кадру для наочного порівняння
    for pred in result.object_prediction_list:
        bbox = pred.bbox.to_xyxy()
        cv2.rectangle(annotated_frame, (int(bbox[0]), int(bbox[1])), (int(bbox[2]), int(bbox[3])), (0, 255, 0), 2)
        cv2.putText(annotated_frame, "SAHI", (int(bbox[0]), int(bbox[1])-5), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2)

    out.write(annotated_frame)

```

```
cap.release()
out.release()
print(f"\n✅ Фінальне відео успішно оброблено та збережено: {output_video_path}")

... Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
Починаю обробку відео: 920 кадрів (2160x3840)
Застосовується A/B тестування на льоту: ByteTrack vs SAHI
0% | 0/920 [00:00<?, ?it/s] requirements: Ultralytics requirement ['lap>=0.5.12'] not found, attempting AutoUpdate...
Using Python 3.12.13 environment at: /usr
Resolved 2 packages in 190ms
Prepared 1 package in 37ms
Installed 1 package in 7ms
+ lap==0.5.13

requirements: AutoUpdate success ✅ 0.7s
WARNING ⚠ requirements: Restart runtime or rerun command for updates to take effect

100% | ██████████ | 920/920 [20:27<00:00, 1.33s/it]
✅ Фінальне відео успішно оброблено та збережено: /content/drive/MyDrive/MKR_Traffic_Project/result_traffic_final.mp4
```

На фінальному етапі модульної контрольної роботи було здійснено тестування навченої моделі YOLOv11 (отримані ваги best.pt) на новому, невідомому для неї відеопотоці роздільної здатності 4K. Головна мета цього етапу — перевірити здатність нейромережі до генералізації, тобто її вміння стабільно розпізнавати автомобілі в реальних умовах (у динаміці), а не лише на статичних зображеннях з валідаційної вибірки.

Для роботи з медіаданими та покадрової обробки відеопотоку було використано бібліотеку комп'ютерного зору OpenCV. Загальний конвеєр (pipeline) інференсу побудовано за таким алгоритмом:

Скрипт ініціалізує відеопотік (cv2.VideoCapture) та зчитує загальні метадані (кількість кадрів, роздільну здатність, FPS).

Кожен кадр послідовно передається у нейронну мережу для розрахунку координат об'єктів (Bounding Boxes) та ймовірності їхнього збігу з класом "car" (Confidence score).

Після накладання візуальної розмітки кадр записується у новий результуючий відеофайл за допомогою cv2.VideoWriter.

Оскільки пряма (покадрова) детекція 4K-відео за допомогою базової моделі YOLO має низку апаратних та архітектурних обмежень (втрата дрібних об'єктів через стиснення тензорів, відсутність пам'яті про об'єкт між кадрами), базовий алгоритм детекції був суттєво модифікований. Для вирішення цих проблем було реалізовано комплекс додаткових інженерних рішень, детальний опис та дослідження яких наведено нижче.

ДОДАТКОВІ ЗАВДАННЯ

1) Відстеження траєкторії об'єктів у русі. Мною був інтегрований алгоритм **ByteTrack**, який успішно відстежує траєкторію та справляється з короткочасними перекриттями об'єктів (occlusions) завдяки передбаченню

руху Фільтром Калмана. Проте, під час дослідження я наочно зафіксував відому проблему трекерів — 'ID Switching'. Вона виникає, коли об'єкт перекривається занадто довго, або відбувається різка зміна освітлення, через що алгоритм втрачає впевненість у траєкторії та ініціалізує старий об'єкт як новий трек

2) Офіційна статистика показує, що точність розпізнавання об'єктів, менших за 32x32 пікселі, різко падає, а об'єкти розміром менше **15-20 пікселів** YOLO просто ігнорує, вважаючи їх візуальним шумом

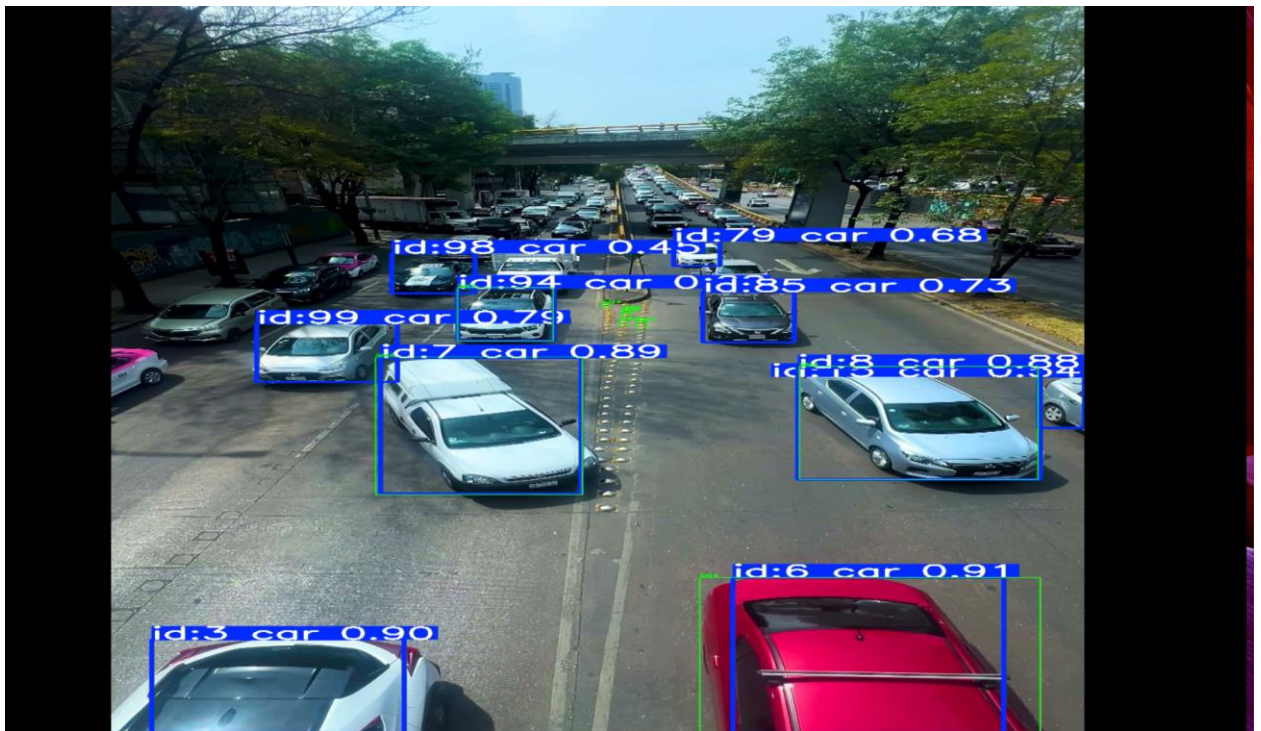
Архітектура YOLOv11n оптимізована для роботи з тензорами розміром 640x640. Коефіцієнт стиснення (downscaling factor) для 4K відео становить рівно 6 ($3840 / 640 = 6$). Мінімальний поріг впевненої детекції об'єкта для базової моделі становить площу близько 20 \ 20 пікселів у нормалізованому просторі мережі.

Без нарізки (Standard Inference): Через коефіцієнт стиснення фізичний розмір автомобіля на оригінальному 4K відео має становити **мінімум 120 \ 120 пікселів**, щоб після стиснення залишитися розпізнаваним для мережі (близько 20 пікселів). Об'єкти меншого розміру втрачають свої візуальні ознаки (features) і сприймаються як фоновий шум.

З використанням нарізки (SAHI Inference): Алгоритм ділить 4K кадр на фрагменти (slices) розміром 640x640 без стиснення оригіналу. Це означає, що коефіцієнт масштабування дорівнює 1. Отже, мінімальний фізичний розмір об'єкта на 4K відео, який здатна знайти модель, знижується до **20 на 20 пікселів**.

Використання алгоритмів нарізки дозволило знизити поріг мінімального розміру розпізнаваного об'єкта у 6 разів (з ~120 пікселів до ~20 пікселів), що критично важливо для виявлення автомобілів на великій дистанції.

3) Повноцінна пакетна обробка послідовних відеокадрів виявилася архітектурно несумісною з одночасною роботою алгоритмів SAHI та ByteTrack через неминуче переповнення відеопам'яті (VRAM) та порушення хронологічної послідовності трекінгу



результат

ВИСНОВОК

У ході виконання модульної контрольної роботи було розроблено комплексну інтелектуальну систему моніторингу дорожнього трафіку на базі архітектури **YOLOv11**.

Головні результати роботи:

Якість моделі: Завдяки етапу кастомної розмітки 795 зображень та подальшій аугментації датасету до 1111 одиниць, було досягнуто високої точності детекції автомобілів (**mAP50 = 87.2%**).

Адаптація до 4K-відео: Реалізація алгоритму **SAHI** (Slicing Aided Hyper Inference) дозволила нівелювати проблему "сліпоти" нейромережі до дрібних об'єктів.

Аналітика руху: Інтеграція трекера **ByteTrack** дозволила системі не просто фіксувати об'єкти, а й вибудовувати їхні індивідуальні траєкторії руху з використанням Фільтра Калмана

Оптимізація: Дослідження апаратної частини показало, що для стабільної роботи системи на роздільній здатності 4K оптимальною є послідовна обробка кадрів із внутрішньою пакетною обробкою фрагментів, що дозволяє ефективно використовувати 16 ГБ відеопам'яті Tesla T4.